



# EAST WEST UNIVERSITY

Department of Computer Science & Engineering

Semester: Spring-2024

Course Code: CSE207

Course Title: Data structure

Section: 01

## Project Report

Project Name: Implementation of Fibonacci heap

### Submitted to

Dr. Maheen Islam

Associate Professor, Department of CSE

### Submitted by

Name: Shawna Akter

ID: 2023-1-60-071

Name: Tabassum Nahar Yeah

ID: 2023-1-60-068

Date of Submission: 28 May, 2024

## Introduction:

A data structure called the Fibonacci Heap allows for quick insert, delete, decrease key, and extract minimum operations when working with a group of objects. This project shows how to create a Fibonacci Heap in C and uses an interactive console interface to illustrate its functioning.

## Implementation Details:

### Fibonacci Heap Algorithm

A Fibonacci heap is a data structure consisting of a collection of trees satisfying the minimum-heap property. It supports a variety of operations that are useful in graph algorithms, particularly those involving shortest paths and minimum spanning trees.

#### #Key Operations

1. Insertion (insert)
2. Extracting the Minimum (extract\_min)
3. Decreasing a Key (decrease\_key)
4. Deleting a Node (delete)

#### #1. Insertion (insert)

##### Algorithm:

1. Create a new node with the given key.
2. Insert this node into the root list of the heap.
3. Update the minimum pointer if necessary.
4. Increment the number of nodes in the heap.

Complexity:  $\mathcal{O}(1)$

#### #2. Extracting the Minimum (extract\_min)

Algorithm:

1. Remove the minimum node (pointed to by the minimum pointer) from the root list.
2. Add its children to the root list.
3. Perform a consolidation step to restructure the heap:
  - Create an array to keep track of roots with the same degree.
  - Traverse the root list and merge trees of the same degree.
4. Update the minimum pointer to point to the new minimum node.
5. Decrement the number of nodes in the heap.

Complexity:  $O(\log n)$  amortized

#3. Decreasing a Key (decrease\_key)

Algorithm:

1. Decrease the key of the given node.
2. If the heap property is violated (the new key is smaller than the parent's key):
  - Cut the node and move it to the root list.
  - Perform a cascading cut if necessary.
3. Update the minimum pointer if necessary.

Complexity:  $O(1)$  amortized

#### #4. Deleting a Node (delete)

Algorithm:

1. Decrease the key of the node to be deleted to  $(-\infty)$ .
2. Extract the minimum node.

Complexity:  $O(\log n)$  amortized

#### # Amortized Analysis

The amortized time for the key operations is derived from the potential function of the Fibonacci heap, which is based on the number of trees in the root list and the number of marked nodes. The potential function helps to average the cost of expensive operations over a sequence of operations, ensuring that the amortized cost remains efficient.

This high-level overview of the Fibonacci heap algorithm provides a comprehensive understanding of its structure and operations, emphasizing its efficiency and usefulness in various applications.

Pseudocode:

#Pseudocode for Fibonacci Heap Operations

#Node and Fibonacci Heap Structures

plaintext

Node:

key

degree

parent

child

left

right

mark

FibonacciHeap:

min

n

#1. Insert

Pseudocode:

plaintext

INSERT(H, x):

if H.min == NULL:

H.min = x

x.left = x

x.right = x

else:

insert x into root list of H

if x.key < H.min.key:

H.min = x

H.n = H.n + 1

#2. Extract Minimum

Pseudocode:

plaintext

EXTRACT\_MIN(H):

z = H.min

if z != NULL:

for each child x of z:

add x to the root list of H

x.parent = NULL

remove z from the root list of H

if z == z.right:

    H.min = NULL

else:

    H.min = z.right

    CONSOLIDATE(H)

    H.n = H.n - 1

return z

CONSOLIDATE(H):

    create an array A of size D(H.n) initialized to NULL

    for each node w in the root list of H:

        x = w

        d = x.degree

        while A[d] != NULL:

            y = A[d]

            if x.key > y.key:

                exchange x and y

        LINK(H, y, x)

        A[d] = NULL

        d = d + 1

    A[d] = x

H.min = NULL

for i = 0 to D(H.n):

    if A[i] != NULL:

        if H.min == NULL:

            create a root list for H containing A[i]

        H.min = A[i]

    else:

        insert A[i] into the root list of H

        if A[i].key < H.min.key:

            H.min = A[i]

LINK(H, y, x):

    remove y from the root list of H

    make y a child of x

    x.degree = x.degree + 1

    y.mark = FALSE

#3. Decrease Key

Pseudocode:

plaintext

DECREASE\_KEY(H, x, k):

    if k > x.key:

        error "new key is greater than current key"

    x.key = k

    y = x.parent

if  $y \neq \text{NULL}$  and  $x.\text{key} < y.\text{key}$ :

CUT(H, x, y)

CASCADING\_CUT(H, y)

if  $x.\text{key} < H.\text{min}.\text{key}$ :

$H.\text{min} = x$

CUT(H, x, y):

remove x from the child list of y

$y.\text{degree} = y.\text{degree} - 1$

add x to the root list of H

$x.\text{parent} = \text{NULL}$

$x.\text{mark} = \text{FALSE}$

CASCADING\_CUT(H, y):

$z = y.\text{parent}$

if  $z \neq \text{NULL}$ :

if  $y.\text{mark} == \text{FALSE}$ :

$y.\text{mark} = \text{TRUE}$

else:

CUT(H, y, z)

CASCADING\_CUT(H, z)

#### #4. Delete Node

Pseudocode:

plaintext

DELETE(H, x):

    DECREASE\_KEY(H, x,  $-\infty$ )

    EXTRACT\_MIN(H)

# User Instructions

1. Insert: Choose option 1 and enter the key to insert.
2. Extract Min: Choose option 2 to extract the minimum key node.
3. Decrease Key: Choose option 3, enter the current key and the new key to decrease.
4. Delete Node: Choose option 4 and enter the key of the node to delete.
5. Display Heap: Choose option 5 to display the heap.

#### Conclusion:

An effective method of managing a priority queue with changeable keys is offered by the implemented Fibonacci Heap. Through an intuitive console interface, the project showcases the fundamental functions and their efficacy.

This extensive project is an excellent learning resource for comprehending complex data structures since it covers both the theoretical and practical applications of Fibonacci Heaps.